



Arab Academy for Science, Technology and Maritime Transport
College of Computing and Information Technology

Course	Operating Systems Security - (CCY4304)
Lecturer	Prof. Dr. Ayman Adel
TAs	Abdelrahman Solyman

12th Project

- **Problem Statement**

- Medical wearables - insulin pumps, pacemakers, and CGMs - run on minimal embedded OSes with **no authentication**, **no file access control**, and **no audit trail**. In 2019, the FDA warned that Medtronic insulin pumps were remotely exploitable precisely because the device kernel had none of these protections. The same vulnerability exists in a vanilla xv6 kernel.

- **Project Goal**

- Extend xv6 with **three** integrated **kernel-level security layers**: role-based user authentication, UNIX-style file permissions, and a syscall audit log simulating the security architecture required by FDA and IEC 62443 for life-critical embedded devices.

- **Roles & Filesystem**

ROLE	UID	ACCESS SCOPE
ADMIN	0	Full system access
PATIENT	1	Own records (read-only)
DOCTOR	2	Dosage logs, patient records

- **Protected Files**

```
/patient/records → uid=1 (read-only)
/dosage/insulin.log → uid=2 (write) / uid=1 (read-only)
/device/config → uid=0 (admin only)
/audit/syscall.log → uid=0 (admin only)
```

Rubric (20 Marks)

Phase 1 - User Authentication (8 marks)

#	Criterion	Description	Marks
1.1	Kernel Data Structures	struct proc extended with uid/gid; /etc/passwd-like file with hashed credentials	2
1.2	System Calls	useradd, userdel, passwd, whoami implemented and functional	2
1.3	Password Security	Passwords hashed before storage; plaintext rejected; correct role assigned on login	2
1.4	Login Enforcement	init spawns login prompt; shell denied without valid credentials	2

Phase 2 - File Access Control (8 marks)

#	Criterion	Description	Marks
2.1	Inode Permission Bits	mode, uid, gid added to struct dinode; medical files pre-assigned correct ownership	2
2.2	chmod / chown Syscalls	Both syscalls work; only owner or admin may change permissions	2
2.3	Kernel Enforcement	fileread, filewrite, exec, open all check permissions; cross-role access blocked	3
2.4	Edge Cases & Testing	Patient blocked from /device/config; doctor writes insulin log; admin bypasses all	1

Phase 3 - Syscall Audit Log (4 marks)

#	Criterion	Description	Marks
3.1	Trap Pretty Printing	trap() maps trapno to readable names; prints PID, UID, trap name, eip to console	1
3.2	Audit Ring Buffer	Persistent kernel ring buffer logging T_SYSCALL traps with PID, UID, tick	1
3.3	Root-Only Reader	audit_read returns EPERM for non-admin; buffer accessible only to uid=0	1
3.4	End-to-End Demo	Log captures failed logins, permission denials, and successful accesses across all roles	1

Bonus Phase - (5 marks)

Create an automated testing program that validates all security requirements and generates a compliance report proving the system meets medical device security standards. This bonus **connects all three phases** and makes grading objective.

#	Criterion	Description	Marks
B.1	Test coverage	Covers all 3 phases (auth, permissions, audit) with minimum 12 test cases	2
B.2	Integration tests	Demonstrates end-to-end security flow (attack → denial → log → detection) through log	1.5
B.3	Report quality	Clear pass/fail output, shows evidence from audit log, exit code indicates success	1
B.4	Automation	Run all tests with single command	0.5

Team size :

- Groups of 2 only.

Deliverables:

- Submit the following on Classroom in **One Combined PDF file**:
 - **PDF Report** that summarizes the steps, commands, screenshot for each step you made through terminal or through GUI and the commands you used through each phase, also attach any source codes or scripts used.
 - Organize your documentation and split your steps based on rubric bullet points.
 - Attach your name/ID on the first page of the report.
- Must push your code to **Github** repo with simple **README** file (stating university name, course code, lecturers)
- Any type of plagiarism will receive a penalty.
- There will be a **discussion** in the 14th Week.